

Assignment 1

1. No, Monitoring systems and CI/CD would need to do health checks automatically without authentication.
2. Liveness checks if the application is alive and would be restarted if it fails. Readiness determines if application is ready to receive traffic. Database health check is better for readiness.

Assignment 2

1. FastAPI is built on Starlette. `HTTPException` is FastAPI's version, while `StarletteHTTPException` comes from the Starlette framework. The Starlette one catches framework errors like 404 and 405.

Assignment 3

1. No. The service layer only contains business logic, so both HTTP and WebSockets can call the same service

Assignment 4

1. JSON is structured, so it's easier for monitoring/automated systems to search, filter, and analyze logs during deployment.
2. Remove any unnecessary logs, use async logging, group logs together, and use centralized logging.

Assignment 5

1. I would not use a complete layered architecture in this case. The extra structure may take more time to implement and maintain than it is worth for a short term and small project.

I would choose the simple version if the project is small and has limited business logic and not expected to grow or be worked on by a large team of people. I would choose the layered architecture if the project is projected to grow or has complex logic and needs to be maintained for a long time

2. No. Since `get_settings()` uses `@lru_cache`, the settings object is created once and reused. Changing the `.env` file while the app is running doesn't automatically update the cached settings and needs a restart.
3. The hybrid query function would be placed inside a separate folder in the service layer outside of the `llm/`, `sql_engine/`, and `rag/` folders. Since the hybrid query combines multiple existing functions, keeping it separate follows single-responsibility and `sql_engine` and `rag` modules can be separate as well.
4. The `request_id` is not guaranteed to propagate to a background thread. We can copy the current context before starting the thread, and run the thread within the context. We can also pass the `request_id` directly to the background thread.

5. Since the LLM is an external application/dependency, any errors with that is a 502 error which is used when there is an error with an intermediary server.

If the LLM's content is filtered because it violates some policy, I would likely use a 400 exception. This is because the user's input is likely the reason the request was rejected by the LLM application. It is possible the LLM itself generates content that violates a policy, it would depend on the specific policy, but I would still count it as 4xx because it's an issue with the user input or generated response rather than server issues.